FluxQueue

Distributed Job Processing System

7-Layer Roadmap — Build it Right

1.5 hrs/day • 10 Weeks • Resume-Ready

#	Layer	Duration	Priority
1	Core System	2 weeks	Must
2	Real-time WebSockets	1.5 weeks	Must
3	Reliability Layer	1.5 weeks	Must
4	Failure Simulation	1 week	High
5	Concurrency + Scale	1 week	High
6	Observability	1 week	High
7	Advanced Feature	1 week	Good

Read the full roadmap before writing any code. Each layer builds on the previous one.

1 Core System

Foundation — get this done fast, no overthinking

What you're building:
Job submission API → Redis queue → Celery worker picks up job → executes → updates status in PostgreSQL.

S1	**Project setup**	Django project + PostgreSQL + Redis. Use Docker Compose to run DB and Redis locally. Install: djangorestframework, celery, redis, psycopg2-binary, simplejwt
S2	**Job model**	Fields: id (UUID), job_type, status (PENDING/RUNNING/COMPLETED/FAILED), payload (JSONB), result (JSONB), retry_count, error_msg, created_at, started_at, completed_at
S3	**JobLog model**	Fields: id, job (FK), level (INFO/WARNING/ERROR), message, created_at. Used to track what happened inside each job
S4	**Submit API**	POST /api/jobs/ — validate input, save job as PENDING to DB, push job ID to Redis via Celery .delay()
S5	**List + Detail API**	GET /api/jobs/ — paginated list. GET /api/jobs/:id/ — full job details with all logs
S6	**Celery task**	execute_job(job_id): fetch job from DB, set RUNNING, run handler, set COMPLETED or FAILED, write logs at each step
S7	**Job handlers**	Write 4 handlers: email_send (sleep 5s + print), pdf_generate (sleep 10s), image_resize (sleep 3s), data_export (sleep 7s). Real logic later
S8	**Test full flow**	Run Django server + Celery worker in separate terminals. Submit job via Postman. Watch it go PENDING → RUNNING → COMPLETED in DB

Goal: Submit a job via Postman, see it execute in background, status updates in DB. Full lifecycle working.

Interview Q: How does the job get from API to worker?
A: API saves job to DB as PENDING, then calls Celery .delay() which pushes the job ID to a Redis list. Worker is running separately with BLPOP on that list — it wakes up, fetches the job ID, queries DB for details, and executes.

2 Real-time System

Replace polling with WebSockets via Django Channels — mandatory upgrade

Why this matters:
Polling (setInterval every 3s) wastes bandwidth and has 3s latency. WebSockets give you instant push updates the moment a job status changes. This is how real production systems work and is rare at intern level.

S1	**Install Django Channels**	pip install channels channels-redis daphne. Add 'channels' to INSTALLED_APPS. Configure ASGI application in asgi.py
S2	**Channel layers config**	In settings.py set CHANNEL_LAYERS to use Redis backend: {'default': {'BACKEND': 'channels_redis.core.RedisChannelLayer', 'CONFIG': {'hosts': [('127.0.0.1', 6379)]}}}
S3	**Job consumer**	Create consumers.py with JobStatusConsumer. On connect: add socket to group 'job_{job_id}'. On disconnect: remove from group
S4	**Worker sends updates**	Inside execute_job() Celery task, after every status change call channel_layer.group_send() to push update to 'job_{job_id}' group
S5	**Log streaming**	Every time you write a JobLog, also send it via channel_layer.group_send() to 'job_{job_id}' group. Frontend receives logs in real time
S6	**Worker health channel**	Create separate 'workers' group. Every 10s, each Celery worker sends its status via group_send to 'workers' group
S7	**URL routing**	In routing.py define WebSocket URL patterns: ws/jobs// and ws/workers/
S8	**React WebSocket client**	Replace all setInterval polling with useEffect + new WebSocket(). On message: parse JSON, update state. On close: show reconnecting

Goal: Open job detail page — status and logs update INSTANTLY when worker processes job. No page refresh, no polling.

Interview Q: Why WebSockets over polling?
A: Polling sends an HTTP request every N seconds regardless of whether anything changed — wasted bandwidth and N-second latency. WebSockets maintain a persistent TCP connection. Server pushes data only when state changes — zero latency, zero wasted requests. For job monitoring where state changes are infrequent but must be shown immediately, WebSockets are the right tool.

3 Reliability Layer

Idempotency + Deep retry + Dead Letter Queue + Timeout — interview gold

Why this matters:
This is what separates someone who built a toy from someone who thinks about production. Every concept here is asked in system design interviews at top companies.

S1	**Idempotency key**	Add idempotency_key field (UUID) to Job model. Before creating a new job, check if key already exists. If yes, return existing job — never create duplicate. Client generates this key
S2	**Atomic job pickup**	Use Redis SET NX EX (set if not exists with expiry) as a distributed lock before executing a job. Only one worker can acquire lock per job_id. Prevents two workers running same job
S3	**Deep retry logic**	Exponential backoff: wait = min(60 * 2**retry_count, 3600). Add jitter: wait += random(0, 30). Cap at 1 hour. Log each retry attempt with reason
S4	**Dead Letter Queue**	After max_retries exceeded, move job to DLQ: set status=DEAD, copy to job_dlq table with failure reason + full error trace. Never delete failed jobs
S5	**DLQ API**	GET /api/jobs/dlq/ — list all dead jobs. POST /api/jobs/dlq/:id/requeue/ — manually requeue a dead job after fixing the root cause
S6	**Timeout handling**	Add timeout_seconds field to Job model. In Celery task use celery.exceptions.SoftTimeLimitExceeded. If job runs longer than timeout, raise exception, mark FAILED, log timeout reason
S7	**Zombie job detection**	Cron job (Celery beat) runs every 5 min: find jobs in RUNNING state with started_at > timeout_seconds ago. These are zombie jobs — worker crashed. Mark them FAILED and requeue
S8	**Test all failure paths**	Test: submit job with duplicate idempotency key (should return same job). Force job to fail 3 times (should land in DLQ). Submit job that times out (should mark FAILED with timeout reason)

Goal: No job runs twice. Failed jobs go to DLQ. Timed-out jobs are detected and handled. All testable via demo.

Interview Q: What is idempotency and why does it matter in job queues?
A: Idempotency means running the same operation multiple times produces the same result. In job queues, if a worker crashes after executing but before acknowledging, the job gets requeued and re-executed. Without idempotency that means two emails sent, two payments charged, two rows inserted. I handle this with a client-provided idempotency key — if the key exists, return the existing job instead of creating a new one.

4 Failure Simulation

Demo failures live in interview — almost no student does this

Why this matters:
Being able to SHOW a failure happening and explain what happens internally is 10x more impressive than just saying 'I handle failures'. Interviewers remember this.

S1	**Chaos mode flag**	Add CHAOS_MODE=True env variable. When enabled, random 30% of jobs will randomly fail mid-execution. Makes it easy to demo failures without manual intervention
S2	**Kill worker mid-job**	Submit a long job (sleep 30s). While RUNNING, kill the Celery worker process (Ctrl+C or kill PID). Watch zombie detection kick in after timeout. Restart worker — job requeues and completes
S3	**Redis down simulation**	Submit 5 jobs. Stop Redis (docker stop redis). Try submitting more — API should return 503 with clear error. Restart Redis — pending jobs resume. Document this behavior
S4	**Force DLQ demo**	Write a job handler that always raises an exception. Submit it — watch it retry 3 times with increasing delays (check logs). After max retries, watch it land in DLQ. Then requeue from DLQ
S5	**Timeout demo**	Set timeout_seconds=5 for a job that sleeps 20s. Submit it — watch it get killed at 5s, marked FAILED with 'timeout' reason. Show this in dashboard
S6	**Duplicate job demo**	Submit same job twice with same idempotency key. Show that only one job was created — second request returned the existing job. Prove idempotency works
S7	**Document all scenarios**	Write a FAILURES.md in your repo documenting each failure scenario, what happens internally step by step, and how the system recovers. This is what you show in interviews

Goal: You can live-demo: kill worker, Redis down, DLQ flow, timeout — and explain each one from first principles.

Interview Q: What happens if a worker crashes mid-job?
A: The job stays in RUNNING state in the DB. My zombie detection cron job runs every 5 minutes — it queries for jobs in RUNNING state where started_at + timeout_seconds < now(). When found, it marks the job FAILED with reason 'worker_crash' and requeues it. Because my handlers are idempotent, re-running a partially executed job is safe.

5 Concurrency + Scaling

Don't just run Celery — understand and SHOW parallel execution

Why this matters:
Anyone can run one Celery worker. Showing you understand concurrency, measuring throughput, and demonstrating how adding workers increases jobs/sec shows systems thinking.

S1	**Multiple workers setup**	Run 3 separate Celery worker processes: celery -A fluxqueue worker --concurrency=4 -n worker1@%h. Each worker gets 4 threads = 12 parallel job slots total
S2	**Measure baseline**	Submit 20 jobs with 1 worker (concurrency=1). Measure total time. Record jobs/sec = 20 / total_seconds
S3	**Scale up and measure**	Repeat with 2 workers, then 4 workers, then 4 workers with concurrency=4. Record jobs/sec each time. Build a simple comparison table
S4	**Show in dashboard**	Add 'Active Workers' count and 'Jobs/sec (last 60s)' metric to your observability dashboard. Calculated from DB: count jobs completed in last 60 seconds
S5	**Concurrency safety**	Verify no two workers execute the same job. Check DB — every job_id should appear in logs exactly once with RUNNING. Your Redis NX lock from Layer 3 handles this
S6	**Queue depth awareness**	Show that when queue depth is high and workers are busy, adding a worker immediately reduces queue depth. Demo this: submit 50 jobs with 1 worker, add second worker, watch queue drain faster

Goal: You can show a table: 1 worker = X jobs/sec, 4 workers = 4X jobs/sec. Linear scaling demonstrated.

Interview Q: How would you scale this to handle 1 million jobs/day?
A: 1M jobs/day = ~12 jobs/sec average. With 4 workers at 4 concurrency each = 16 slots. Each job takes ~5s average = 3.2 jobs/sec per slot = 51 jobs/sec total — more than enough. For true scale: add workers horizontally (they're stateless), use Redis Cluster for the queue, shard PostgreSQL by date for query performance, and add separate queues per job type so heavy jobs don't block fast ones.

6 Observability

Clean dashboard that screams production mindset

Why this matters:
Observability is how you know your system is healthy in production. A clean dashboard that shows the right metrics instantly signals you think beyond just making things work.

S1	**Stats API**	GET /api/stats/ returns: total_jobs, pending_count, running_count, completed_count, failed_count, dead_count, avg_execution_time_ms, jobs_per_minute, queue_depth
S2	**Queue depth over time**	Store queue_depth snapshots every 30s in a queue_metrics table (timestamp, depth). Stats API returns last 60 snapshots. Frontend renders as Recharts LineChart
S3	**Job state breakdown**	Pie or bar chart: PENDING / RUNNING / COMPLETED / FAILED / DEAD counts. Updates via WebSocket when any job changes state
S4	**Worker health panel**	Each worker shown as a card: name, status (ACTIVE/IDLE/OFFLINE), jobs_processed, last_heartbeat. Worker goes OFFLINE if no heartbeat in 30s
S5	**Failure rate metric**	failure_rate = (failed + dead) / total * 100. Show as percentage with color: green < 5%, yellow 5-20%, red > 20%
S6	**Avg execution time**	Calculated as: AVG(completed_at - started_at) for COMPLETED jobs in last hour. Show per job_type breakdown so you can see which job type is slowest
S7	**Job throughput chart**	Line chart: jobs completed per minute over last 60 minutes. Shows system throughput over time. Spikes visible when you added more workers
S8	**Make it clean**	Use Tailwind for layout. Dark mode. Cards with subtle shadows. Color-coded status badges. Numbers big and readable. This is what you screenshot for your resume

Goal: Dashboard shows all metrics live via WebSocket. Anyone can understand system health in 5 seconds by looking at it.

Interview Q: How do you know if your system is healthy?
A: I track 5 key signals: queue depth (growing queue = workers overwhelmed), failure rate (> 5% means something is wrong), avg execution time (spikes mean downstream slowness), worker heartbeats (missing heartbeat = dead worker), and jobs/min throughput (drop = bottleneck). All shown live on the dashboard via WebSocket push.

7 Advanced Feature

Priority Queues (recommended) — adds depth without blowing scope

Why priority queues:
Priority queues are the most practical advanced feature. They directly demonstrate understanding of real production systems — critical jobs (payment processing) should not wait behind batch export jobs. Celery has built-in support, so implementation is clean and not over-engineered.

S1	**Define 3 queues**	Create 3 Celery queues: high_priority, default, low_priority. In Celery config: task_routes maps each job_type to its queue. Email = high, PDF = default, data_export = low
S2	**Priority field**	Job model already has priority (1-5). Map to queues: 4-5 = high_priority, 2-3 = default, 1 = low_priority. Set when job is submitted
S3	**Worker queue assignment**	Start workers with queue flags: celery worker -Q high_priority,default,low_priority --prefetch-multiplier=1. Workers drain high_priority queue first
S4	**Dedicated workers**	Optionally: run 1 dedicated worker for high_priority only. This guarantees critical jobs never wait behind low priority jobs even under heavy load
S5	**Demo priority**	Submit 10 low_priority jobs (they start processing). Then submit 1 high_priority job. Show it completes before the low priority jobs even though they were submitted first
S6	**Show in dashboard**	Dashboard shows 3 queue depth bars: high / default / low. Color coded. Interviewers can visually see priority separation

Goal: High priority jobs always execute before low priority jobs. Demonstrable live with queue depth dashboard.

Interview Q: How do you ensure critical jobs don't wait behind batch jobs?
A: I use separate Celery queues per priority level: high_priority, default, low_priority. Workers are configured to drain high_priority queue first. For truly critical workloads, I run a dedicated worker that only consumes high_priority — this guarantees critical jobs are never blocked by a backlog of batch exports, regardless of queue depth.

Final Resume Description

FluxQueue — Distributed Job Processing System

A production-grade async job processing platform built with Django, Celery, Redis, Django Channels, and React.

- Designed task queue architecture with Redis broker and concurrent Celery workers; replaced polling with Django Channels WebSockets for real-time job status and log streaming
- Implemented full reliability layer: idempotency keys preventing duplicate execution, exponential backoff retry with jitter, Dead Letter Queue for exhausted jobs, and timeout handling with zombie job detection
- Built failure simulation suite: live demos of worker crash recovery, Redis downtime handling, and DLQ requeue flow — each scenario documented with internal behavior explanation
- Demonstrated horizontal scaling: measured jobs/sec throughput with 1 vs 4 workers; implemented priority queues ensuring critical jobs execute before batch jobs under load
- Shipped real-time observability dashboard: queue depth, failure rate, avg execution time per job type, worker heartbeat monitoring, and throughput charts via WebSocket push

10-Week Timeline at 1.5 hrs/day

Week	Layer	Done When
1–2	Layer 1: Core System	Jobs submitted via API, execute in background, full lifecycle in DB
3–4	Layer 2: WebSockets	Status and logs update live in browser, no polling anywhere
5–6	Layer 3: Reliability	Idempotency, DLQ, timeout, zombie detection all tested
7	Layer 4: Failure Simulation	Can demo crash, Redis down, DLQ flow live
8	Layer 5: Concurrency	Throughput table showing linear scale with workers
9	Layer 6: Observability	Clean dashboard, all metrics live, looks production-ready
10	Layer 7: Priority Queues	High priority jobs demonstrably skip the queue. Deploy + README done

Rule: Don't move to the next layer until the current one works and you can explain every part of it cold.